

SOLID Principles

Design Principles and Approaches

- S** Single Responsibility
- O** Open/Closed
- L** Liskov substitution
- I** Interface Segregation
- D** Dependency Inversion

SoftUni Team
Technical Trainers



Table of Contents

1. Single Responsibility
2. Open/Closed
3. Liskov Substitution
4. Interface Segregation
5. Dependency Inversion



sli.do

#fund-csharp

Why Clean Code Matters?

- How **clean code** (or its absence) **affects** our software?

"...So if you want to go **fast**,
if you want to get done **quickly**,
if you want your code to be **easy to write**,
make it **easy to read**."

– Robert C. Martin



Single Responsibility

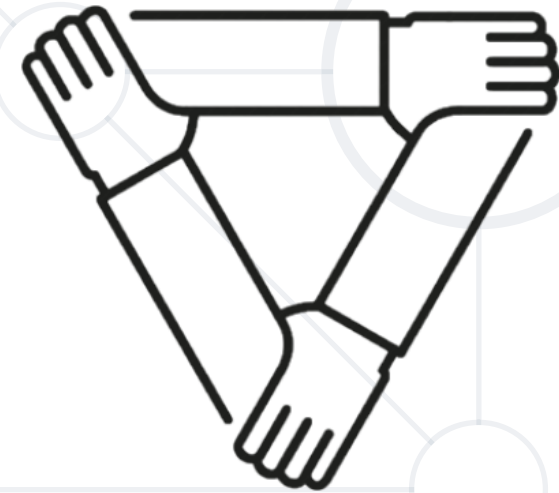
What is Single Responsibility?

- Every class should be responsible **for only a single part of the functionality** and that responsibility should be entirely **encapsulated** by the class.

"There should never be more than one reason for a class to change."
- Robert C. "Uncle Bob" Martin



- **Cohesion** refers to the grouping of **functionally related processes** into a particular module.
- Aim for **strong cohesion**
 - Each **task** maps a **single** code unit
 - A method should do **one operation**
 - A class should represent **one entity**



Strong Cohesion / Loose Coupling (2)

- **Coupling** – the degree of dependence between modules
 - How closely connected two modules are
 - The strength of the relationship between modules
- Aim for **loose** coupling
 - Supports **readability** and **maintainability**
 - Often a sign of good system **design**

- **Small number** of instance variables inside a class
- Each method of a class should manipulate **one or more** of those variables
- Two modules should **exchange** as **little information** as possible
- Creating an **easily reusable** subsystem



Open/Closed

What is the Open/Closed Principle?

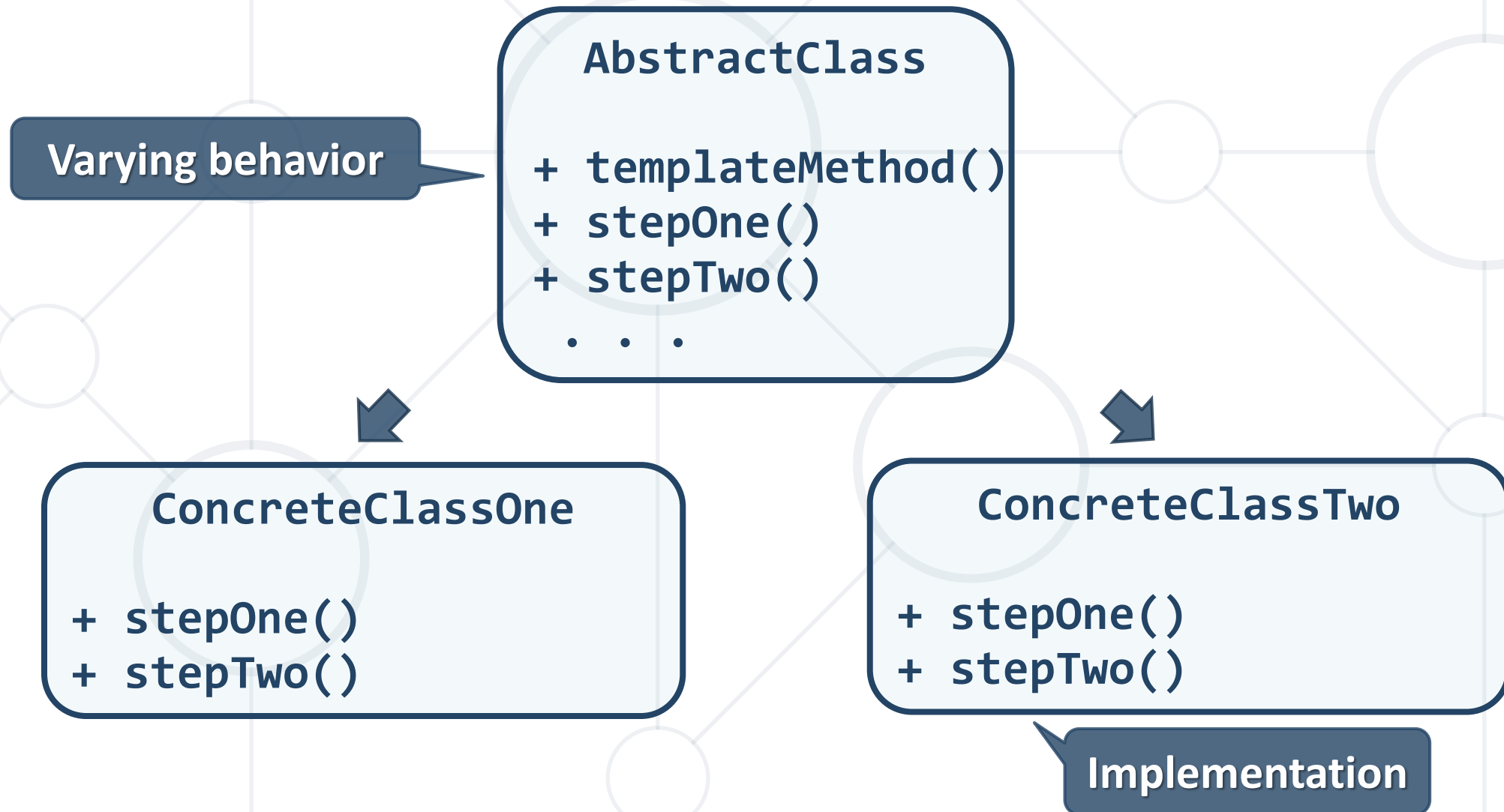
- Software entities like **classes**, **modules** and **functions** should be **open** for **extension**, but **closed** for **modifications**
- **Extensibility**
 - Adding a new behavior **doesn't require** changes over existing source code
- **Reusability**
 - subsystems are **suitable for reusing** in other projects – modularity

- Need to **retest** after changes
 - Old parts changed -> possible bugs
- **Cascading changes** through modules
- Logic depends on **conditional statements** (“IS-A” checking)

- Parameters
 - Control behavior specifics via a **parameter** or a **delegate**
- Rely on abstraction, **not implementation**
 - Inheritance / Template Method Pattern
- Strategy Pattern
 - Plug in model (insert a new implementation of the interface)

- By experience – know the problem domain and if a **change** is very **likely** to **recur**
- New domain problem – implement the **most simple** way
 - Changes once – **modify**, second time – **refactor**
- TANSTAAFL – There Ain't No Such Thing As A Free Lunch
 - OCP adds **complexity** to design
 - No design can be **closed against all changes** - know which ones to guard

Template Method Pattern



Template Method Pattern (2)

```
public abstract class CrossCompiler
{
    public void CrossCompile()
    {
        this.CollectSource();
        this.CompileToTarget();
    }

    protected abstract void CollectSource();
    protected abstract void CompileToTarget();
}
```

Template method

Template Method Pattern (3)

```
public class iPhoneCompiler : CrossCompiler
{
    protected override void CollectSource()
    protected override void CompileToTarget()
    { //iphone specific compilation }
}
```

```
public class AndroidCompiler : CrossCompiler
{
    protected override void CollectSource()
    protected override void CompileToTarget()
    { //android specific compilation }
}
```

Problem: Stream Progress Info

- Refactor the skeleton given for this problem:
 - **StreamProgressInfo** class must work with Music class too
 - Make sure that if you add a new class, which provides **BytesSent** and **Length**, getters will work without touching **StreamProgressInfo**

Solution: Stream Progress Info

```
public class StreamProgressInfo
{
    private IStreamable streamable;
    public StreamProgressInfo(IStreamable streamResult)
    {
        this.streamable = streamResult;
    }
    public int CalculateStreamProgress()
    {
        return (this.streamable.BytesSent * 100) /
                this.streamable.Length;
    }
}
```

Solution: Stream Progress Info (2)

```
public interface Istreamable
{
    int Length { get; }
    int BytesSent { get; }
}
```

```
public class File : Istreamable
{
    //TODO: Add business logic
}
public class Music : Istreamable
{
    //TODO: Add business logic
}
```

- Refactor the skeleton given for this task so that:
 - **GraphicEditor** class draws **all kinds of shapes** without asking what kind of shape we pass
 - Be sure if you add a **new type** of shape, the system will still work correctly **without** touching **GraphicEditor**

Solution: Graphic Editor

```
public interface IShape
{
    void Draw();
}
public class Rectangle : IShape
{
    public void Draw()
    { Console.WriteLine("I'm Rectangle"); }
}
public class GraphicEditor
{
    void DrawShape(Shape shape)
    { shape.Draw(); }
}
```

Make the same
for Circle



Liskov Substitution

LSP - Substitutability

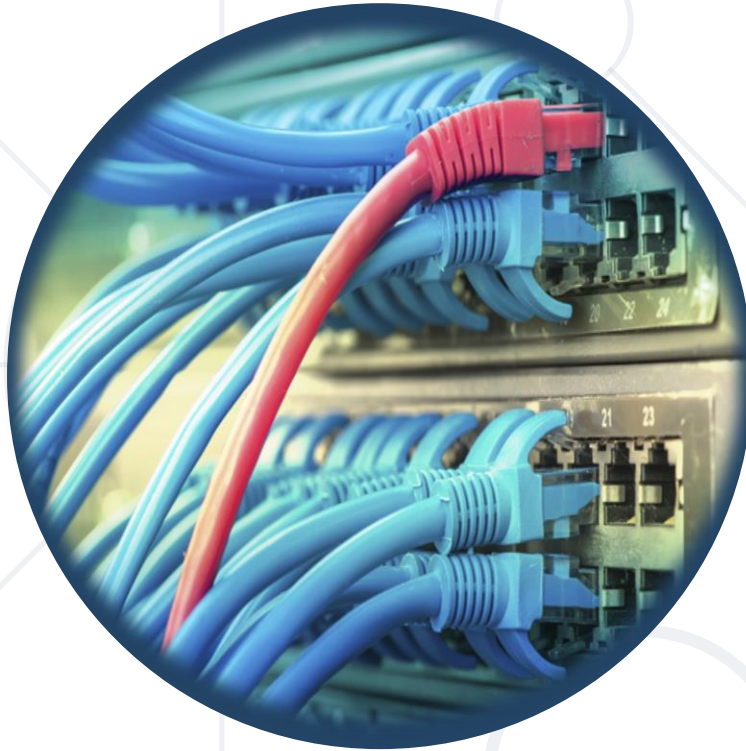
- 
- Derived types must be completely **substitutable** for their base types
 - Derived classes
 - only **extend** functionalities of the base class
 - must **not** remove **base** class **behavior**

Student **IS-SUBSTITUTED-FOR** Person

- Type Checking
- Overridden methods say "I am not implemented"
- Base class depends on its subtypes



- Tell Don't Ask
 - If you need to check what is the object - move the behavior **inside the object**
- New Base Class – if **two classes** share a common behavior, but are not substitutable, create a third, from which **both derive**
- There **shouldn't** be any **virtual methods** in constructors



Interface Segregation

What is Interface Segregation?

- Segregate interfaces
 - Prefer **small, cohesive** (lean and focused) interfaces
 - Divide "**fat**" interfaces into "**role**" interfaces

"**Clients** should not be forced to depend on methods they do not use."

- Agile Principles, Patterns and Practices in C#



- Classes whose interfaces are not cohesive have "fat" interfaces

```
public interface IWorker
{
    void Work();
    void Sleep();
}
```

```
public class Robot : IWorker
{
    void Work() { ... }
    void Sleep()
    { throw new NotImplementedException() }
}
```

- **Not implemented** methods
- A Client references a class, but only uses a **small portion** of it

"Abstraction is **elimination**
of the **irrelevant** and
amplification of the **essential**."
– Robert C. Martin

- What does the client **see** and **use**?
- The "**fat**" interfaces implement a **number of small** interfaces with just what you need
- All **public members** of a class divided in **separate classes**
 - again, could be thought of as an interface
- Let the **client define interfaces** – "**role**" interfaces

- Small and Cohesive "Role" Interfaces

```
public interface IWorker
{
    void Work();
}
public interface ISleeper
{
    void Sleep();
}
public class Robot : IWorker
{
    void Work() { // Do some work... }
}
```


- Problem that the **Adapter pattern** solves
 - **Reusing** classes that do not have an **interface** that a client requires
 - Making classes with **incompatible** interfaces work together
 - Providing **an alternative** interface for a class

- Convert the **incompatible** interface of a class Adaptee into another interface – Target, that clients require

```
class Adaptee
{
    public void SpecificRequest()
    {
        Console.Write
            ("Called SpecificRequest()");
    }
}
```

```
interface Target
{
    void Request();
}
```

- Define a separate class – Adapter, that does the job

```
class Adapter : Target
{
    private Adaptee adaptee = new Adaptee();
    public void Request()
    {
        // Possibly do some other work
        adaptee.SpecificRequest();
    }
}
```

- **Refactor** the skeleton given for this task so that
 - **DetailsPrinter** class prints correctly any kind of employee, which is in the collection
 - Remove any "**is**" statements from your code
 - Be sure if you add a new type of employee, the system will work correctly without touching **DetailsPrinter**

```
public class Employee
{
    private string name;
    public Employee(string name)
    {
        this.name = name;
    }
    public override string ToString()
    {
        return $"Name: {this.name}";
    }
}
```

Solution: Detail Printer (2)

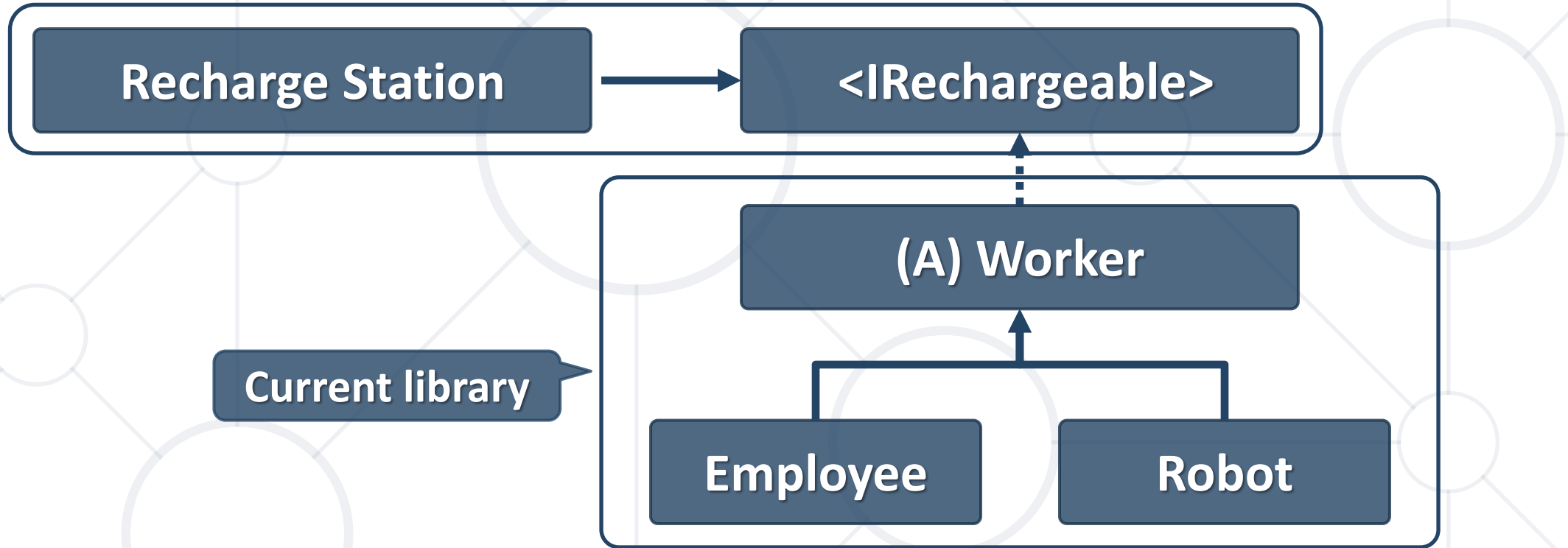
```
public class Manager : Employee
{
    public Manager(string name,
        ICollection<string> documents) : base(name)
    {
        this.Documents = new List<string>(documents);
    }
    public IReadOnlyCollection<string> Documents { get; set; }
    public override string ToString()
    {
        return base.ToString() + $"Documents:
            {string.Join(Environment.NewLine, this.Documents)}";
    }
}
```

Solution: Detail Printer (3)

```
public class DetailsPrinter
{
    private IList<Employee> employees;
    public DetailsPrinter(IList<Employee> employees)
    {
        this.employees = employees;
    }
    public void PrintEmployees()
    {
        foreach(Employee employee in employees)
            Console.WriteLine(employee);
    }
}
```

Problem: Recharge

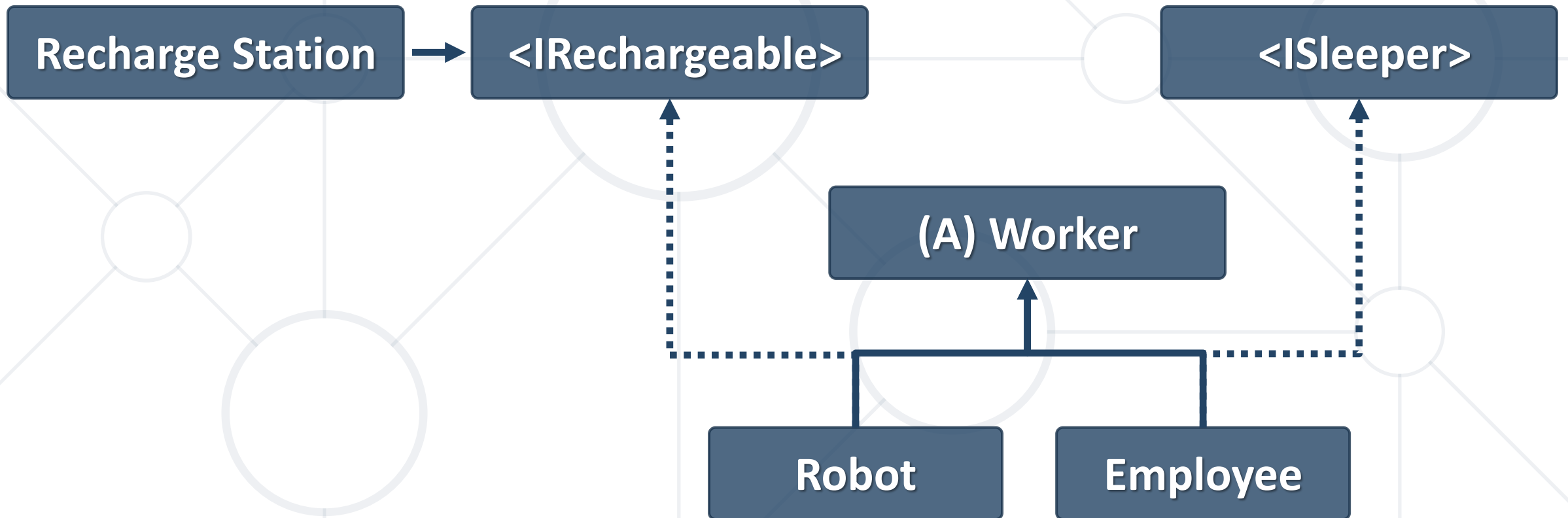
- You are given some classes



- Refactor the code, so that it **conforms to ISP**
- Consider the case that you **don't own the library**

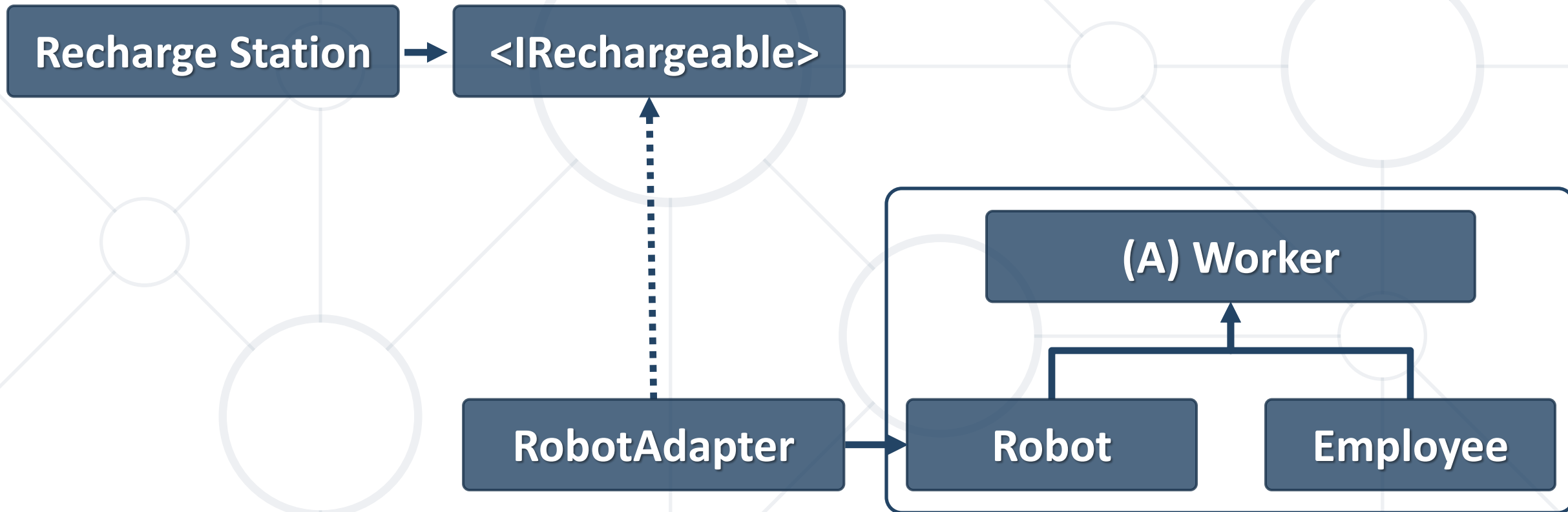
Solution: Rechargeable

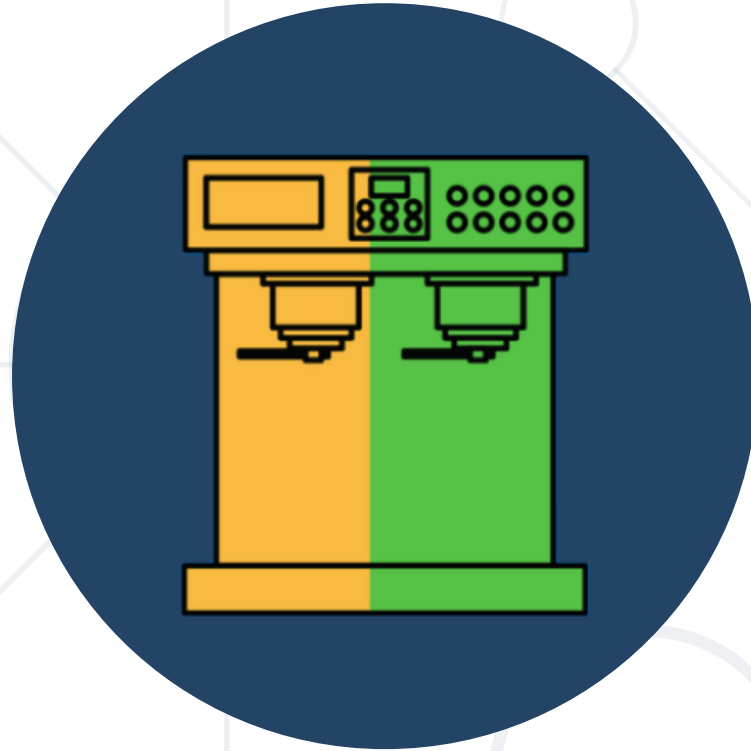
- Multiple Inheritance (If you own the library)



Solution: Rechargeable (2)

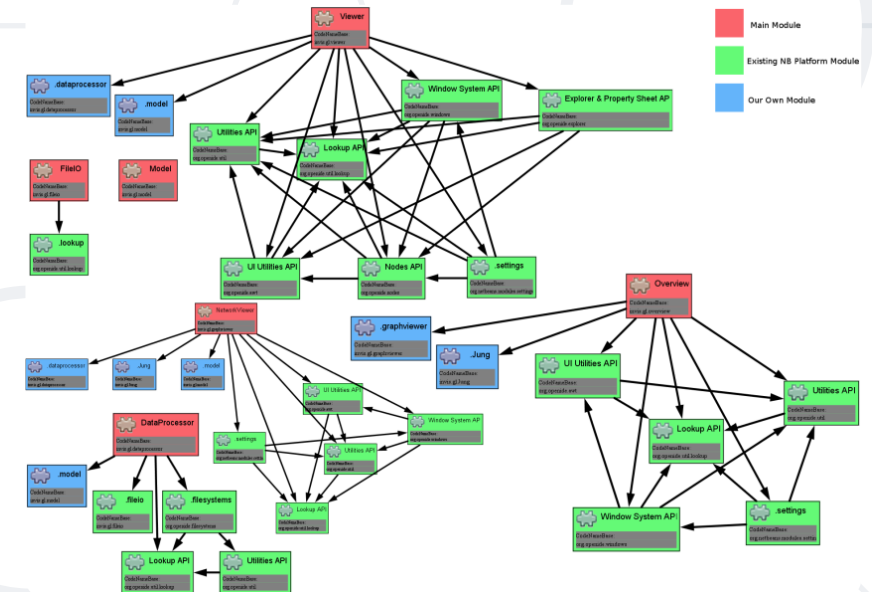
- Adapter Pattern (If you don't own the library)





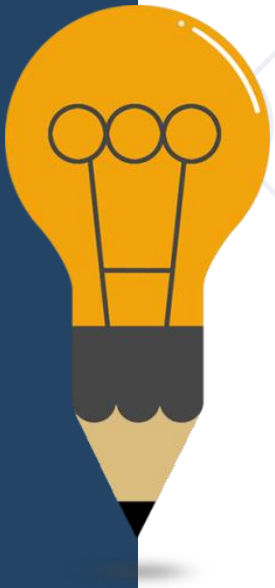
Dependency Inversion

- Depend on abstractions

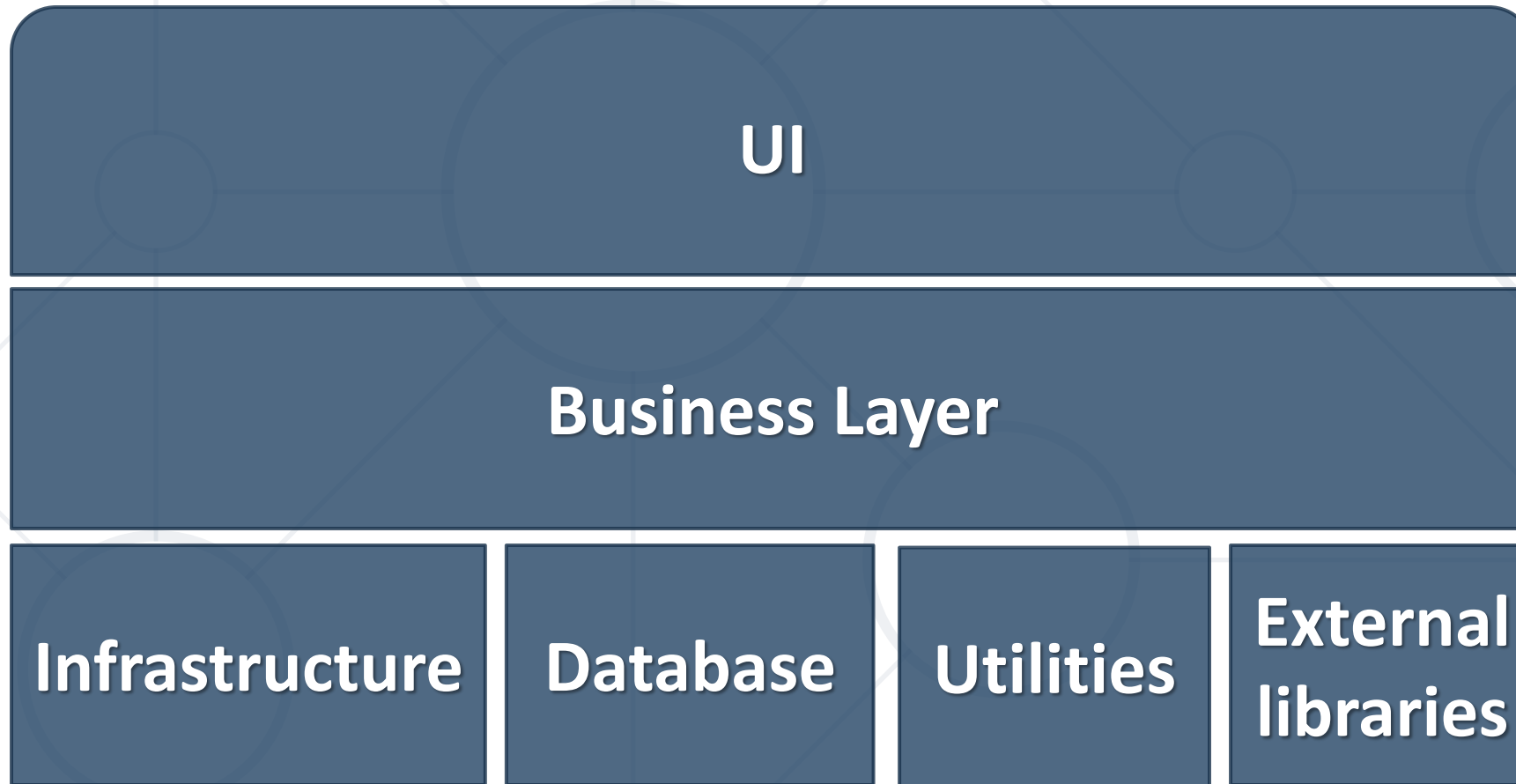


Dependency Examples

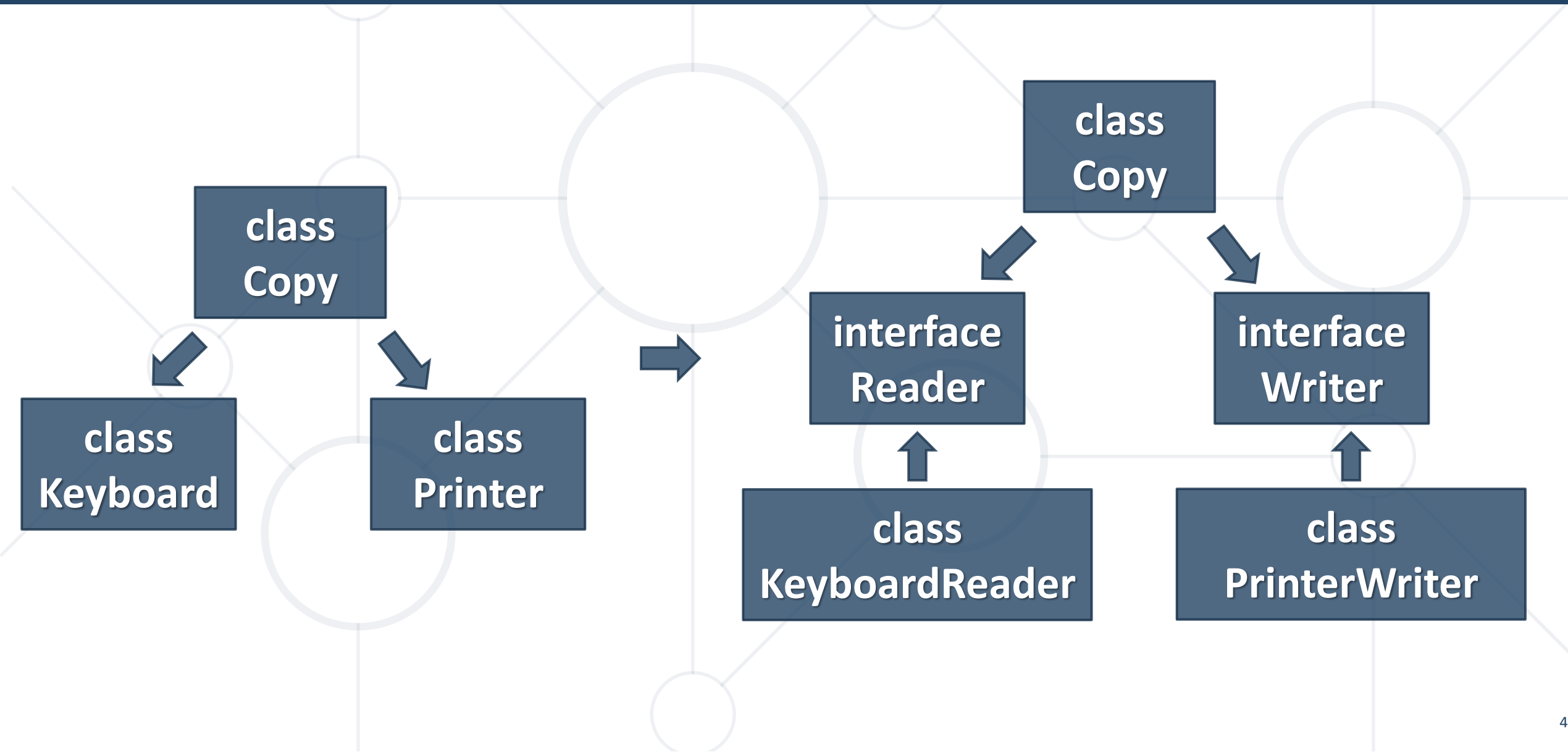
- A **dependency** is any external component / system:
- Framework
- 3rd party library
- Database
- File system
- Email
- Web service
- System resource (e.g. clock)
- Configuration
- The **new** keyword
- Static method
- Global function
- Random generator
- Console



Dependencies in Traditional Programming



Depend on Abstractions



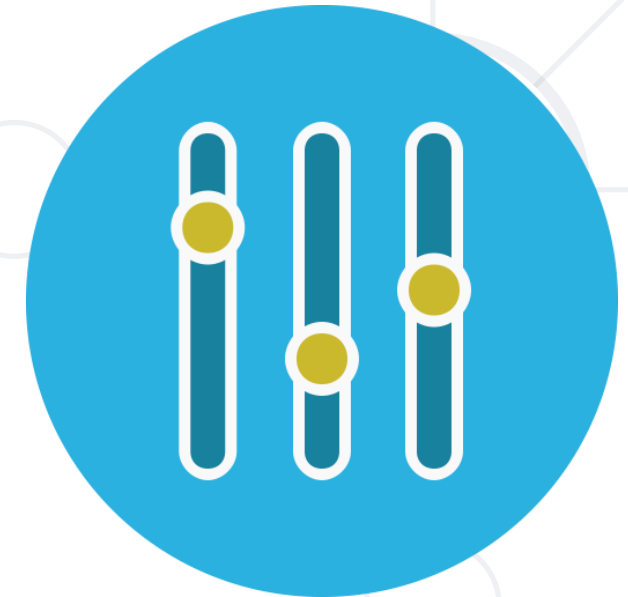
Types of Dependency Injection



**Constructor
injection**



**Property
injection**



**Parameter
injection**

Constructor Injection – Pros and Cons

■ Pros

- Class' requirements are self-documenting
- We don't have to worry about state validation

■ Cons

- Too many parameters
- Sometimes, the functionality doesn't need all of the dependencies



Constructor Injection - Example

```
class Copy
{
    private IReader reader;
    private IWriter writer;
    public Copy(IReader reader, IWriter writer)
    {
        this.reader = reader;
        this.writer = writer;
    }
    // Read/Write data through the reader/writer
}

var copy = new Copy(new ConsoleReader(),
                    new FileWriter("out.txt"));
```

Property Injection – Pros and Cons

■ Pros

- Functionality can be changed at any time
- That makes the code very flexible

■ Cons

- State can be invalid
- Less intuitive to use



Property Injection - Example

```
class Copy
{
    public IReader Reader { get; set; }
    public IWriter Writer { get; set; }
    public void CopyAllChars(reader, writer)
    {
        // Read/Write data through the reader/writer
    }
}

Copy copy = new Copy();
copy.Reader = new ConsoleReader();
copy.Writer = new FileWriter("output.txt");
copy.CopyAllChars();
```

Parameter Injection – Pros and Cons

- Pros

- Changes are only localized to the method

- Cons

- Too many parameters
- Breaks the method signature



Parameter Injection - Example

```
class Copy
{
    public CopyAllChars(IReader reader, IWriter writer)
    {
        // Read/Write data through the Reader/Writer
    }
}

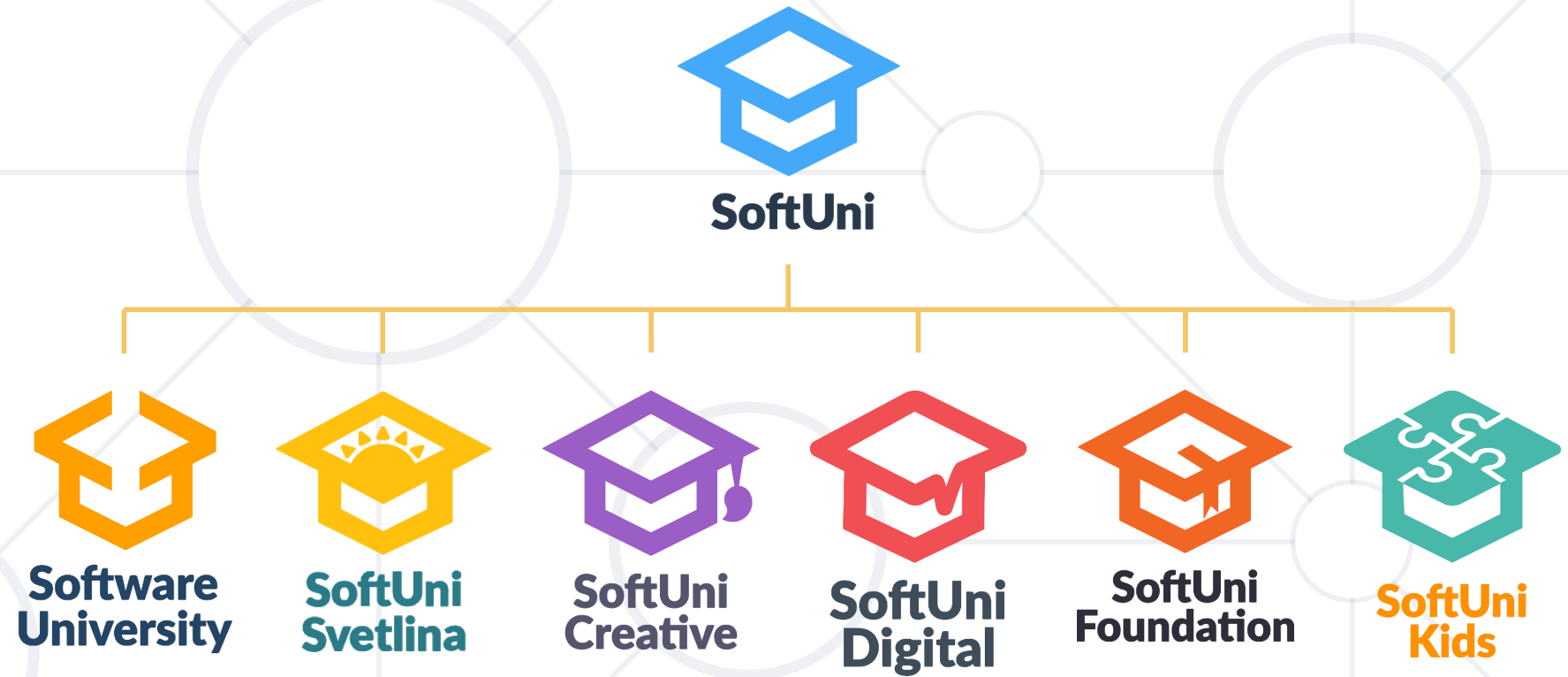
Copy copy = new Copy();
var reader = new ConsoleReader();
var writer = new FileWriter("output.txt");
copy.CopyAllChars(reader, writer);
```

- Classic DIP Violations:
 - Using the **new** keyword
 - Using **static** methods / properties
- How to fix code, that violates the DIP:
 - **Extract interfaces** + use **constructor injection**
 - Set up an Inversion of Control (**IoC**) container

- SOLID principle make software more:
 - Understandable
 - Flexible
 - Maintainable



Questions?



SoftUni Diamond Partners



XSsoftware



SBTech
we know sports



telenor



SoftwareGroup
doing it right

NETPEAK



SmartIT



Postbank

Решения за твоето утре

**SUPER
HOSTING**
.BG

INDEAVR

Serving the high achievers



INFRAGISTICS®

LIEBHERR



æternity



codexio

SoftUni Organizational Partners



OneBit
SOFTWARE



Trainings @ Software University (SoftUni)

- Software University – High-Quality Education and Employment Opportunities
 - softuni.bg
- Software University Foundation
 - <http://softuni.foundation/>
- Software University @ Facebook
 - facebook.com/SoftwareUniversity
- Software University Forums
 - forum.softuni.bg



- This course (slides, examples, demos, videos, homework, etc.) is licensed under the "Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International" license

